LAB: GPIO Digital InOut(eval board)

LAB: GPIO Digital InOut

Date: 2026-09-20

Author/Partner: Haeun Yang

Demo Video: [EC 26-2] LAB (LAB_GPIO_DIO_LED_Photosensor)

[EC 26-2] LAB (LAB_GPIO_DIO_multiLED)

Introduction

In this lab, you are required to create a simple program that toggles multiple LEDs using a push-button input.

You must create your own drivers for GPIO digital input and output control.

Requirement

Hardware

- MCU
  - NUCLEO-F411RE
- Eval Board
- Actuator/Sensor/Others:

Software

- PlatformIO, CMSIS, EC_HAL library

Problem 1: Create EC_HAL library

Function Definitions

ecGPIO2.h

```
void GPIO_init(PinName_t pinName, int mode);
void GPIO_write(PinName_t pinName, int Output);
uint32_t  GPIO_read(PinName_t pinName);
void GPIO_mode(PinName_t pinName, int mode);
void GPIO_ospeed(PinName_t pinName, int speed);
void GPIO_otype(PinName_t pinName, int type);
void GPIO_pupd(PinName_t pinName, int pupd);
```

ecGPIO2.c

GPIO_init()

지정한 핀의 클럭을 활성화하고 GPIO 모드를 설정하는 초기화 함수.

```
void GPIO_init(PinName_t pinName, uint32_t mode)
```

Parameters

- `pinName` : 초기화할 핀 이름 (예: `PA_5`)
- `mode` : GPIO 모드 (`INPUT` , `OUTPUT` , `AlterFunc` , `Analog` 중 하나)

```
void GPIO_init(PinName_t pinName, uint32_t mode){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);

    if (Port == GPIOA)
        RCC_GPIOA_enable();
    if (Port == GPIOB)
        RCC_GPIOB_enable();
    if (Port == GPIOC)
        RCC_GPIOC_enable();
    if (Port == GPIOD)
        RCC_GPIOD_enable();
    if (Port == GPIOE)
        RCC_GPIOE_enable();
    if (Port == GPIOH)
        RCC_GPIOH_enable();

    GPIO_mode(pinName, mode);
}
```

Example code

```
GPIO_init(PA_5, OUTPUT);
```

GPIO_write()

지정한 핀에 HIGH 또는 LOW 값을 씀.

```
void GPIO_write(PinName_t pinName, uint32_t Output)
```

Parameters

- `pinName` : 값을 쓸 핀 이름
- `Output` : 출력할 값 (`0` = LOW, `1` = HIGH)

```
void GPIO_write(PinName_t pinName, uint32_t Output){
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);

    Port->ODR &= ~(1UL<<pin) ;
    Port->ODR |= (Output<<pin) ;
}
```

Example code

```
GPIO_write(PA_5, HIGH);   // PA_5 핀을 HIGH로 설정
GPIO_write(PA_5, LOW);    // PA_5 핀을 LOW로 설정
```

GPIO_read()

지정한 핀의 현재 입력 전압 상태(0 또는 1)를 읽는다.

```
uint32_t GPIO_read(PinName_t pinName)
```

Parameters

- `pinName`: 읽어올 핀 이름

```
uint32_t GPIO_read(PinName_t pinName){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    unsigned int bit;
    bit = (Port->IDR>>pin) & 1 ;
    return bit;
}
```

Example code

```
uint32_t state = GPIO_read(BUTTON_PIN);
```

GPIO_mode()

핀의 동작 모드(입력/출력/대체기능/아날로그)를 설정.

```
void GPIO_mode(PinName_t pinName, uint32_t mode)
```

Parameters

- `pinName` : 설정할 핀 이름
- `mode` : `00` =Input, `01` =Output, `10` =Alternate Function, `11` =Analog

```
// GPIO Mode : Input(00), Output(01), AlterFunc(10), Analog(11)
void GPIO_mode(PinName_t pinName, uint32_t mode){
   GPIO_TypeDef * Port;
   unsigned int pin;
   ecPinmap(pinName,&Port,&pin);

   Port->MODER &= ~(3UL<<(2*pin));     // Clear '00'
   Port->MODER |= mode<<(2*pin);       // Set mode
}
```

Example code

```
GPIO_mode(PA_5, OUTPUT);
```

GPIO_ospeed()

핀의 출력 속도(slew rate)를 설정.

```
void GPIO_ospeed(PinName_t pinName, uint32_t speed)
```

Parameters

- `pinName` : 설정할 핀 이름
- `speed` : `00` =Low, `01` =Medium, `10` =Fast, `11` =High speed

```
// GPIO Speed : Low speed (00), Medium speed (01), Fast speed (10), High speed
(11)
void GPIO_ospeed(PinName_t pinName, uint32_t speed){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    Port->OSPEEDR &= ~(3UL<<(2*pin)) ;    // Clear '00'
    Port->OSPEEDR |= speed<<(2*pin) ;

}
```

Example code

```
GPIO_ospeed(PA_5, MEDIUM_SPEED);
```

GPIO_otype()

핀의 출력 타입(Push-Pull 또는 Open-Drain)을 설정.

```
void GPIO_otype(PinName_t pinName, uint32_t type)
```

Parameters

- `pinName` : 설정할 핀 이름
- `type` : `0` =Push-Pull, `1` =Open-Drain

```
// GPIO Output Type: Output push-pull (0, reset), Output open drain (1)
void GPIO_otype(PinName_t pinName, uint32_t type){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    Port->OTYPER &= ~(1UL<<(pin)) ;    // Clear '00'
    Port->OTYPER |= ~(type<<pin) ;
}
```

Example code

```
GPIO_otype(PA_5, PUSH_PULL);
```

GPIO_pupd()

핀의 풀업/풀다운 저항 설정을 지정.

```
void GPIO_pupd(PinName_t pinName, uint32_t pupd)
```

Parameters

- `pinName` : 설정할 핀 이름
- `pupd` : `00` =No pull, `01` =Pull-up, `10` =Pull-down, `11` =Reserved

```
// GPIO Push-Pull : No pull-up, pull-down (00), Pull-up (01), Pull-down (10),
Reserved (11)
void GPIO_pupd(PinName_t pinName, uint32_t pupd){
    GPIO_TypeDef * Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    Port->PUPDR &= ~(3UL<<(2*pin)) ;    // Clear '00'
    Port->PUPDR |= pupd<<(2*pin) ;
}
```

Example code

```
GPIO_pupd(PA_5, PU);
```

Problem 2: Toggle a single LED with Digital Sensor(Photodetector)

Problem

Toggle the LED by covering the photodetector sensor.

- Dark (LED ON), Bright (LED OFF) and repeat
- Connect the Photodetector sensor to PA_0 pin on Nucleo Board.

Configuration

Function	Port - Pin	Configuration
Photodetector on JKIT	PA_0	Digital IN, Pull-Up
LED0	PB_12	Digital OUT, Pull-UP, Open-Drain, Medium Speed

Code

```
#include "ecRCC2.h"
#include "ecGPIO2.h"

#define LED_PIN PB_12
#define BUTTON_PIN PA_0

// Initialization
void setup(void){
    RCC_HSI_init();
    GPIO_init(BUTTON_PIN, INPUT) ;  // Initialize the pushbutton pin as an
input:
    GPIO_pupd(BUTTON_PIN, PU) ;

    GPIO_init(LED_PIN, OUTPUT) ;   // Initialze the LED pin as an output
    GPIO_ospeed(LED_PIN, MEDIUM_SPEED) ;
    GPIO_pupd(LED_PIN, PU) ;
    GPIO_otype(LED_PIN, OPEN_DRAIN) ;
}

int main(void){
    setup();
    int buttonState = 0 ;

    while(1){
```

```
        // check if the pushbutton is pressed. Turn LED on/off accordingly
        // toggle (Turn on -> off -> on -> off)
        buttonState = GPIO_read(BUTTON_PIN) ;
        if(buttonState == 1)  GPIO_write(LED_PIN, LOW) ;
        else GPIO_write(LED_PIN, HIGH) ;
        delay_ms_HSI(100) ;
    }
}
```

Problem 3: Toggle multiple LEDs with a button

Problem

Use the four LEDs (LED0 to LED3) on the JKIT evaluation board.

Each time Button B1 is pressed, turn on one LED at a time in a sequential loop

- e.g., LED0 -> LED1 -> LED2 -> LED3 -> LED0)

Configuration

Function	Port - Pin	Configuration
SW2 on JKIT	PA_0	Digital IN, Pull-Up
LED_0~LED_3	PB_12, PB_13, PB_14, PB_15	Digital OUT, Pull-UP, Open-Drain, Medium Speed

Code

```
#include "ecRCC2.h"
#include "ecGPIO2.h"

#define LED_0 PB_12
#define LED_1 PB_13
#define LED_2 PB_14
#define LED_3 PB_15

#define BUTTON_PIN PA_4

// Initialization
void setup(void) ;
void LEDsetup(uint32_t LED) ;


int main(void){
    setup() ;
    int buttonState = 1 ;
    int prevButtonState = 1 ;
    int flag = 0 ;
```

```
    while(1){
        // check if the pushbutton is pressed.
        // change like: LED0 -> LED1 -> LED2 -> LED3 -> LED0 (Active high)
        buttonState = GPIO_read(BUTTON_PIN) ;
        if (buttonState == 0 && prevButtonState == 1) {
            if (flag == 0){
                GPIO_write(LED_0, HIGH) ;  // BUTTON -> Active Low(0 일 때가 눌렸
을 때)
                GPIO_write(LED_1, LOW) ;
                GPIO_write(LED_2, LOW) ;
                GPIO_write(LED_3, LOW) ;
                flag = 1 ;
            }
            else if (flag == 1){
                GPIO_write(LED_0, LOW) ;  // BUTTON -> Active Low(0 일 때가 눌렸을
때)
                GPIO_write(LED_1, HIGH) ;
                GPIO_write(LED_2, LOW) ;
                GPIO_write(LED_3, LOW) ;
                flag = 2 ;
            }
            else if (flag == 2){
                GPIO_write(LED_0, LOW) ;  // BUTTON -> Active Low(0 일 때가 눌렸을
때)
                GPIO_write(LED_1, LOW) ;
                GPIO_write(LED_2, HIGH) ;
                GPIO_write(LED_3, LOW) ;
                flag = 3 ;
            }
            else if (flag == 3){
                GPIO_write(LED_0, LOW) ;  // BUTTON -> Active Low(0 일 때가 눌렸을
때)
                GPIO_write(LED_1, LOW) ;
                GPIO_write(LED_2, LOW) ;
                GPIO_write(LED_3, HIGH) ;
                flag = 0 ;
            }
        }
        prevButtonState = buttonState ;
        delay_ms_HSI(100) ;
    }
}

void setup(void){
    RCC_HSI_init();
    GPIO_init(BUTTON_PIN, INPUT) ;  // Initialize the pushbutton pin as an
input:
    GPIO_pupd(BUTTON_PIN, PU) ;

    LEDsetup(LED_0) ;
    LEDsetup(LED_1) ;
    LEDsetup(LED_2) ;
    LEDsetup(LED_3) ;
}

void LEDsetup(uint32_t  LED){
    GPIO_init(LED, OUTPUT) ;   // Initialze the LED pin as an output
    GPIO_pupd(LED, PU) ;
```

```
    GPIO_otype(LED, PUSH_PULL) ;
    GPIO_ospeed(LED, MEDIUM_SPEED) ;
}
```

Results

Demo Video: [EC 26-2] LAB (LAB_GPIO_DIO_LED_Photosensor)

```
                    [[EC 26-2] LAB (LAB_GPIO_DIO_multiLED)]
(https://youtube.com/shorts/6T6pQTptimw)
```

Discussion

1. **Find out a typical solution for software debouncing and hardware debouncing.**

Ordinarily, this bounce has little or no effect on the circuit, but if the digital circuit is fast enough to detect and respond to multiple bounces, there can be serious consequences. An engineer's task is to avoid or mitigate the effects of this bounce, or "debounce" the switch.

Hardware way focuses on the root cause: the electrical noise. It gurantees the digital signal won't have such noise during transitions. It does it through the use of low-pass filters.

One of the simplest hardware-based switch debounce solutions employs a resistor-capacitor (RC) network in conjunction with an SPST switch.

In the case of an SPDT switch, a common hardware debounce solution is to employ an SR latch. Ever since companies like IBM used this technique for the switch panels on their mainframe computers circa the 1960s, this approach has been regarded as the *crème de la crème* of simple hardware debounce solutions. Such a latch can be formed using two back-to-back two-input NAND gates.

Software way is about ignorint these false positives on the signal transitions to determine which ones are the real events we're looking for, and which ones aren't. There are two main aprroaches.

First is periodic button state sampling. The rule is simple: if there are X-number of consecutive samples with a changed state, we assume the transition really happened. This periodic sampling is often in the order of 10 to 25-ms and is commonly paced by a hardware timer to guarantee fixed intervals and to free some CPU usage.

Second is tracking edge interrupts with minimum cooldown. This works by coordinating the detection of edge changes with the spacing between these: there must be a minimum duration before legitimate signal transition. This cooldown phase is commonly implemented through a timer, which kicks-off on the edge-interrupts: the timer gets restarted on each transition and only when it expires (after 10-ms to 25-ms of no edge-changes), the firmware handles the transition as an event.

2. **What method of debouncing did this NUCLEO board use for the push-button(B1)?**

NUCLEO board does not use hardware debouncing; instead, software debouncing was implemented.

In the "Toggle Multiple LEDs with a Button" lab, the following line was used inside the `while` loop:

```
delay_ms_HSI(100) ;
```

This function forces a 100 ms pause after handling one event, before the next call to `GPIO_read`. Even if the button contacts generate a bouncing signal for a few milliseconds at the moment of the press, that noise is entirely absorbed within the 100 ms delay window, so any subsequent bouncing is never sampled. As a result, a single press is prevented from being counted as multiple events.

This approach is based on the periodic sampling method mentioned earlier, though it is a simplified, rather than a formal, implementation. A proper implementation of periodic sampling would read the input multiple times and verify that N consecutive readings return the same value before accepting the transition as valid.

In addition, a `prevButtonState` variable was used to store the button's previous state, allowing the program to detect only the exact moment of the falling (or rising) edge — that is, the instant the button transitions from unpressed to pressed — rather than reacting continuously while the button remains held down.

Reference

https://blog.golioth.io/how-to-debounce-button-inputs-in-a-real-time-operating-system/

https://www.digikey.com/en/articles/how-to-implement-hardware-debounce-for-switches-and-relays